Week 2: Mixed effects models

Emma James

1 Introduction

> **Learning objectives**
>
> As a reminder, the learning objectives for this week are:
>
> 1. Describe some limitations of ANOVA and regression
> 2. Explain the benefit and implications of modelling across groups
> 3. Define and identify appropriate fixed versus random effects
> 4. Evaluate the benefits and challenges of a mixed effects approach
> 5. Fit, evaluate, and interpret mixed effects models using the *lme4* package

In the lecture this week we learned how mixed effects models can account for different sources of clustering in data. We saw how random effects can be used to model variation in the intercept and slope terms for different groups, and how the fitting algorithms can partially mitigate influences of imbalanced data

Today, we'll put this into practice in modelling both participant- and item-level variation in two different datasets. We will start by using the `sleepstudy` dataset, a frequently used example dataset loaded by the *lme4* package. We will then progress to a psycholinguistic dataset collected here in the York Psychology Department, which will involve modelling multiple sources of clustering simultaneously.

---

2 Setting up

2.1 Packages

Today's practical will use the following packages. You will need to install each one, before loading them at the top of your script.

```
library(tidyverse)    # for data wrangling
library(lme4)         # for model fitting
library(lmerTest)     # significance tests
library(lattice)      # for plotting random effects
library(performance)  # for checking model assumptions
```

2.2 Scientific notation

You might find it easier to interpret the model output if you turn off scientific notation. You can do that with the following line of code:

```
options(scipen=999)
```

Note that this just affects the *presentation* of very small values, not the values themselves.

---

3 Part A: Core example

3.1 Introduction to the dataset

We will start with a built-in dataset from the *lme4* package: the `sleepstudy` dataset. This dataset was taken from a study by Belenky et al. (2003), who were interested in the effects of sleep deprivation and recovery on people's performance. This version of the dataset includes data from participants who had only three hours of sleep per night during the study. They completed a psychomotor vigilance task each day, which provided an average reaction time score for each participant as they responded to a visual stimulus.

Start by pulling the dataset into your environment.

```
sleep_dat <- sleepstudy
```

You will see the dataset contains 180 observations across 3 variables: `Subject` (participant ID), `Days` (the day of the observation, starting at 0), and `Reaction` (the mean reaction time).

Use your preferred R code to answer the following questions about the dataset:

- How many unique participants do the observations come from?
- How long was this phase of the study in days?
- What was the slowest and fastest reaction time?

```
# There are several ways you could do these, depending on your preference and style! Some example

# Number of participants
length(unique(sleep_dat$Subject))

## ...or we could do this using tidyverse code for consistency, e.g.,
sleep_dat %>%
  distinct(Subject) %>%
  count()

# How long was this phase of the study in days?
```

```r
max(sleep_dat$Days)  # but careful, this can be misleading if we consider...
min(sleep_dat$Days)  # ...showing how important it is to explore data properties!
length(unique(sleep_dat$Days))  # an alternative answer

# Range of reaction times
range(sleep_dat$Reaction)
```

3.2 Traditional analyses

We want to test the hypothesis that the length of sleep deprivation is positively associated with participants' reaction times (i.e., the more you are sleep deprived, the slower you are).

This is an association between two continuous variables. If we were to ignore that the data include repeated observations from each participant, we could think of this as a regression analysis. Let's start by running one here:

```r
# Specify model
lm_mod <- lm(Reaction ~ 1 + Days, data = sleep_dat)

# Print output
summary(lm_mod)
```

The model formula specifies that the dependent variable (`Reaction`) is predicted by (`~`) both an intercept (`1`) and our independent variable `Days`.[1]

Take a look at the output. The `Estimate` for the `(Intercept)` tells you the predicted reaction time when `Days` = 0, the first day of the study. The model then estimates that each of the `Days` is significantly associated with ~10.46 ms increase in reaction time.

However, as we noted above, this violates a core assumption of linear regression: that the data are independent. This is clearly not the case as there are multiple observations per participant in the study. We can therefore use random effects in our model to account for this.

3.3 Random intercepts model

Let's start by adding a random intercept for each participant, this time using the `lmer()` function from the *lme4* package. The model formula is very similar to the one above, but now we add a random intercept (`1`) per participant as follows:

```r
# Specify model
ri_mod <- lmer(Reaction ~ 1 + Days + (1|Subject), data = sleep_dat)

# Print output
summary(ri_mod)
```

[1] Note that we don't actually need to specify the intercept (1), as the model will include this by default. However, we've done so here to make clear the similarities in structure when adding random effects below.

Reassuringly, you should see that the `Fixed effects` coefficients remain very similar to the estimates we saw in our standard regression model. However, the error associated with these estimates is lower, as some of this variance can be attributed to between-subject differences captured by the random effects.

You should also take a look at the `Random effects` section of the output. It tells us the number of observations in the model, and how many "groups" there were in any given random effect (here, Subject) - this is a good way of checking nothing has gone awry! The output then tells us how much variance is associated with each one.

We can dig a bit deeper into the estimated random effects if we want to find out more about the groups in our study. For example, we can extract participants' relative intercepts (i.e., deviations from the fixed intercept) to find out who started the study the fastest or slowest. Run the code below: which Subject IDs to these map on to?

```
# Extract random intercepts
subj_int <- ranef(ri_mod)

# Plot dot plot
dotplot(subj_int)
```

3.4 Adding random slopes

So far we have modelled variation in the intercept for each participant (i.e., their start point when `Days` = 0), but assumed that the effect of `Days` is the same for everybody. Let's add a random slope to the model to consider variation in this too.

```
# Specify model
rs_mod <- lmer(Reaction ~ 1 + Days + (1+Days|Subject), data = sleep_dat)

# Print output
summary(rs_mod)
```

Where is there most variance: in the random intercepts, or the random slopes?

Try extracting and plotting the random effects coefficients as you did above. Is there any sign of a correlation between how fast participants were when they started and how much they changed over time? We would see this if the random slopes followed a similar (or opposite) pattern to the intercept.

```
# Extract random intercepts
subj_re <- ranef(rs_mod)

# Plot dot plot
dotplot(subj_re)
```

3.5 Testing model assumptions

Like any statistical model, its use depends on a number of assumptions—primarily which concern the error terms (i.e., residuals). The `performance` package provides some excellent tools to inspect these, which we will use today.

Two assumptions concern the ***linearity*** of relationship between the variables, and the spread of variance across the range (i.e., ***homoscedasticity***). We can inspect these by plotting the model fitted values against the residuals as follows:

```
# Check linearity and heteroscedasticity
check_model(rs_mod, check = "linearity")
```

You should see that the reference line is fairly flat (at least within the confidence intervals), indicating that a linear relationship is appropriate. There looks to be slightly higher variance towards the higher end of the distribution, suggesting that the model is doing slightly less well at predicting higher values.

Another important assumption is the ***normality of residuals:*** we want to ensure there is not systematic bias that would undermine statistical inference from the model. Let's inspect the distribution:

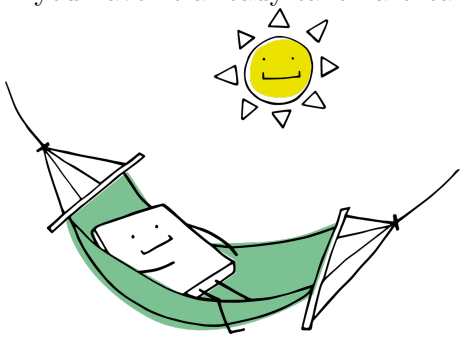
```
# Normality of residuals
check_model(rs_mod, check = "normality")

# Inspect further via QQ plot
check_model(rs_mod, check = "qq")
```

The two plots reveal issues with heavy tails in the distribution, indicative of some more extreme values that are not well-captured by the model. This is perhaps unsurprising considering that we are modelling reaction time data, which often deviates from a normal distribution. We'll return to this issue with a larger dataset in Part B.

Take a break!

If you haven't already taken a break today, we suggest you do so before you start the next section.



---

4 Part B: Apply your knowledge

4.1 Introduction to the dataset

Let's move on to a different example, this time from an experiment with two different conditions (i.e., a factorial design). You'll remember from the lecture that mixed effects models first became popular in psycholinguistics, where researchers want to generalise their findings beyond their specific sample of people *and* the specific words that they selected to include in their experiment.

We will use a psycholinguistic dataset today, collected by Adam Curtis (a former York PhD student) in Professor Gareth Gaskell's lab. They were interested in how recent linguistic encounters support us in comprehending subsequent words. For example, hearing "*The entertainer filled the balloon from the gas cylinder and inhaled it to make her voice squeaky"* might later bias our representation of"balloon" towards particular aspects of its meaning (e.g., *helium*) and make them easier to process. Their experiment looked for this meaning priming over a window of ~10 minutes.

We will use a subset of the data today, but you can read more about their study here if you are interested. The full data and analysis scripts are all available on the Open Science Framework, and we can import the data file directly. Use the following code to select only the relevant conditions from Experiment 1, and variables of interest for today's analysis.

```
# Download and preprocess original dataset
prime_dat <- read_csv("https://osf.io/e5vpn/?action=download") %>%
  filter(exp == 1) %>%
  filter(condition == "unprimed" | condition == "consistent") %>%
  select(id, condition, target, probe, correct, rt)
```

The dataset includes the following variables:

- **id:** a unique identifier for each participant
- **condition***:* consistent priming or unprimed condition[2]
- **target:** the target word whose meaning was/was not recently primed in a sentence context (depending on condition)
- **probe:** the related word for the semantic judgement task
- **correct:** whether the two words were correctly judged as related in meaning (1) or not (0)
- **rt:** response time in milliseconds

Use R code of your choice to answer the following questions about the dataset:

- How many participants took part in the study?
- How many target items were there?
- What was the average (mean) accuracy for each condition?
- What was the average (mean) RT for each condition?
- What does the distribution of RTs look like?

[2]Curtis et al. also included a third condition with an *inconsistent prime* that biased meaning away from the target, but we have excluded that for the purposes of today's demonstration.

```r
# These are some examples only, there are several ways to get these answers...

# Number of participants and words
prime_dat %>%
  summarise(n_ppts = length(unique(id)),
            n_words = length(unique(target)))

# Averages
prime_dat %>%
  group_by(condition) %>%
  summarise(mean_acc = mean(correct),
            mean_rt = mean(rt))

# RT distribution
hist(prime_dat$rt)
```

4.1.1 Data formatting

We have two further formatting tasks to do before we can begin our analysis.

We want to test whether people are faster to judge the target and probe words as related if they have recently seen a consistent prime (relative to an unprimed condition). This means that we only want to analyse RTs from correct answers, as the reaction times for errors likely reflect a different process. Filter the dataset for correct answers only, and assign it to a new object for use:

```r
correct_dat <- filter(prime_dat, correct == 1)
```

Note that for this model out independent variable (i.e., predictor) is a *factor*—a categorical variable that in this case has two levels. We want it to treat the "unprimed" condition as the baseline or reference group. Create a new formatted version of this variable, called `condition_f` which should be a factor version of the existing `condition` variable. We write out the two levels to specify that they should be considered in this order (rather than the default alphabetical interpretation).

```r
correct_dat <- mutate(correct_dat,
                      condition_f = factor(condition,
                                           levels = c("unprimed", "consistent")))
```

What are the mean RTs per condition now that you have filtered for accuracy?

```r
correct_dat %>%
  group_by(condition) %>%
  summarise(mean_rt = mean(rt))
```

4.2 Traditional analyses

4.2.1 T-test of participant averages

We can't run a t-test on the dataset as it is because it violates the assumption of independence: we have multiple responses from each participant. Use the `group_by()` and `summarise()` functions to calculate an average (i.e., mean) response time for each participant in each condition. Call your new data frame `ppt_means`, and reformat it into wide format (i.e., one row per participant, with separate columns containing the average for each condition).

> Click for a hint!
>
> Run `help(pivot_wider)` to inspect the help files for reshaping your dataset. You will need to specify your new column names using `names_from`, and the values that should go in them using `values_from`.

```
ppt_means <- correct_dat %>%       # take the existing data frame
  group_by(id, condition_f) %>%    # for each id and condition
  summarise(mean_rt = mean(rt)) %>%  # calculate the mean rt
  pivot_wider(names_from = condition_f, values_from = mean_rt)
```

Use the `t.test()` function to run a *paired* t-test. Use the internet or the help files to help you if you are unsure. Your output should look as follows:

```
    Paired t-test

data:  ppt_means$consistent and ppt_means$unprimed
t = -3.6171, df = 65, p-value = 0.0005827
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -31.605607  -9.119575
sample estimates:
mean difference
      -20.36259
```

4.2.2 T-test of word averages

The t-test on participant means tells us that that condition effect likely generalises to other people in the population we have sampled from. What if we also want to know whether it generalises to other words that we might have chosen?

Repeat the steps above but for items instead of participants: create a set of *item* means by calculating an average RT for each target and condition. Then run a paired t-test. Are the results consistent with the participant version?

```r
# Calculate item means
item_means <- correct_dat %>%          # take the existing data frame
  group_by(target, condition_f) %>%   # for each item and condition
  summarise(mean_rt = mean(rt)) %>%   # calculate the mean rt
  pivot_wider(names_from = condition_f, values_from = mean_rt)  # long to wide

# Run paired t-test
t.test(item_means$consistent, item_means$unprimed, paired = TRUE)
```

4.3 Random intercepts model

Mixed effects models build on the by-participant/by-item averaging approach by allowing us to account for both participant *and* item variation in the same model. We can use the original trial-level data so that we are not wasting information by averaging.

Start by fitting a random intercepts model: you want to predict the `rt` variable by your formatted condition variable (`condition_f`). Include random intercepts for `id` and `target`.

```r
#|
prime_ri <- lmer(rt ~ condition_f + (1|id) + (1|target), data = correct_dat)
summary(prime_ri)
```

Call `summary()` on your model and ask yourself the following questions about the output:

- Check the number of observations, as well as the numbers associated with your grouping levels (id, target). Do they match up to your exploration of the dataset?
- What the model-estimated reaction time for responses associated with the *unprimed* condition? Remember, we formatted our condition variable so that the unprimed condition was the reference level (0).
- Does the prime condition significantly affect reaction times? How many ms does it change by, and in which direction?
- Do participants or items show the most variance?

Use the `dotplot()` function on the random effects (`ranef()`) of the model like we did above. You can use the plot window arrows to toggle between the two types of random effect. Which participant IDs and target words were fastest (-) and slowest (+) overall? To what extent do these correspond to the participant and item means you calculated for the t-tests?

4.4 Adding random slopes

We should also consider adding random slopes to model variation in condition effects. Do some participants/words show larger priming effects than others?

Try adding random slopes for the effect of condition, for both participant ID and target items. Try plotting the random effects. What do you notice about the confidence intervals, and how the condition estimates are ordered relative to the intercept?

> Tip!
>
> You might want to increase the size of the plot window so that you can see the plots more quickly.

```
# Fit maximal model
prime_rs <- lmer(rt ~ condition_f + (1 + condition_f|id) +
             (1 + condition_f|target), data = correct_dat)

# Inspect output
summary(prime_rs)

# Plot random effects
dotplot(ranef(prime_rs))
```

4.5 Model selection

The random effect variation for condition effects across participants looks surprisingly small, and the model output shows that they are perfectly negatively correlated with the intercept. The clue here is in the warning that was printed during model fitting: *boundary (singular) fit: see help('isSingular')*. Essentially, this means that the model could not be estimated properly.

In this situation, we would typically remove random effects components until the warning goes away. Try this—can we fit any random slopes at all? There is not a consensus on the best approach to deciding which random effects to include/remove from more complex models, but you can read more about this debate via the suggested reading for this week.

4.6 Testing model assumptions

Once you have decided on your final model (i.e., one without singularity issues), use the `check_model()` functions from the *performance* package to check for linearity, heteroskedasticity, and the normality of residuals, as we did in Part A.

```
# Check linearity and heteroscedasticity
check_model(prime_ri, check = "linearity")

# Check normality of residuals
check_model(prime_ri, check = "normality")
check_model(prime_ri, check = "qq")
```

Sometimes a simple transformation can fix problems with skew, such as a log-transform. Try fitting another model, but use the log function to transform the dependent variable within the model formula (`log(rt)`). Inspect the residuals as above: what do you think of this new model?

```r
# Fit model
log_mod <- lmer(log(rt) ~ condition_f + (1|id) + (1|target), data = correct_dat)

# Inspect residuals
check_model(log_mod, check = "normality")
check_model(log_mod, check = "qq")
```

---

5 Part C: Challenge yourself!

In today's examples, we fitted a linear mixed effects model on reaction time data, a continuous variable.

Mixed effects models offer an extremely flexible framework, which we can also use to model accuracy data (0, 1). The underlying maths is different, but the process of fitting them in R is largely the same.

Using the `prime_dat` file you initially loaded (before we filtered for correct responses), try to figure out how to analyse the accuracy data for this experiment. You will need to find the function for running a *binomial* mixed effects model, and specify the family as you run it.

```r
# Format condition factor
prime_dat <- prime_dat %>%
  mutate(condition_f = factor(condition,
                              levels = c("unprimed", "consistent")))

# Binomial model - start with maximal model - singularity issues
prime_acc1 <- glmer(correct ~ condition_f + (1 + condition_f|id) +
             (1 + condition_f|target), family = "binomial", data = prime_dat)

# Model with slopes for participant only - no issues
prime_acc2 <- glmer(correct ~ condition_f + (1 + condition_f|id) +
             (1|target), family = "binomial", data = prime_dat)

# Check - model with slopes for items only - singularity issues
prime_acc3 <- glmer(correct ~ condition_f + (1|id) +
             (1+ condition_f|target), family = "binomial", data = prime_dat)

# Inspect output for most complex model that was successful fit
summary(prime_acc2)

# Check model
check_model(prime_acc2)
```

---

6 Summary

6.1 Recap

Today we've seen how mixed effects models can allow us to model repeated observations from participants within a regression framework. We also saw how this approach can be used to model multiple sources of variation simultaneously, providing a single overarching estimate of the effects of interest in the context of this variation. This makes it a very flexible framework that can be used across a wide range of research contexts.

During the practical, we also encountered some of the challenges associated with this technique. Mixed effects models remain subject to a number of assumptions—as per any statistical approach—which are important to check and may require further consideration. We also saw that we might not be able to estimate all the model components we set out to, which is a topic widely debated in the literature.

6.2 Further learning

This week's core reading includes an excellent tutorial paper by Violet Brown. If you want to gain more interactive experience in fitting mixed effects models in R, there is also a relevant chapter on DataCamp.

! Feedback!

These practical sessions are new this year. Please take a moment to rate the difficulty and length of these practical activities via this survey, and let us know of any issues you encountered or aspects you didn't understand (positive feedback is also welcome!). You will need to be logged in with your university account to respond, but your submission will be anonymous.
For more general questions about the practical, please post them on the VLE Discussion Board.